Comparative Performance and Computational Complexity Analysis of Pre-Trained Deep Learning Models for Citrus Fruit Disease Classification

Nur E Jannatul Farjana
Dept. of Computer Science and Engineering
International University of Business Agriculture and Technology
Uttara, Dhaka-1230, Bangladesh

Abdullah Miraz
Dept. of Computer Science and Engineering
International University of Business Agriculture and Technology
Uttara, Dhaka-1230, Bangladesh

Krishna Das
Dept. of Computer Science and Engineering
International University of Business Agriculture and Technology
Uttara, Dhaka-1230, Bangladesh

ABSTRACT

Citrus fruit diseases cause significant yield loss in commercial orchards, making early and reliable identification essential for farmers. This paper evaluates four pre-trained deep convolutional neural networks (MobileNetV2, InceptionV3, ResNet50, and VGG19) for classifying citrus diseases while examining the computational trade-offs required for field deployment. Training and evaluation were conducted on 1,463 RGB images covering four distinct classes: canker, black spot, greening, and healthy fruit. Across the experimental benchmarks, MobileNetV2 delivered the best overall performance, reaching 97.29% test accuracy and a 98.54% F1-score, while requiring only 406.29 s of training time and a compact 9.82 MB memory footprint. InceptionV3 achieved comparable accuracy at 96.59% but required more memory (22.81 MB). ResNet50 struggled under frozen-base transfer learning, recording a test accuracy of 67.34% and an F1-score of 58.21%, whereas VGG19 demanded substantial computational resources (3360.14 s training time and 118.59 MB footprint). These findings confirm that lightweight depthwise separable architectures provide an optimal balance of high diagnostic accuracy and low hardware overhead for mobile and edge-based field diagnosis.

General Terms

Algorithms, Experimentation, Performance

Keywords

Transfer Learning, Convolutional Neural Networks, Citrus Disease Classification, Performance Metrics, Computational Complexity, MobileNetV2

1. INTRODUCTION

Agriculture is one of the most vital sectors in Bangladesh as a large part of the economy depends on it directly. Citrus fruit production has seen rapid growth over the past five decades, expanding from 19,163 tonnes in 1973 to 175,325 tonnes in 2022 [1]. But citrus orchards in regions like Sylhet, Moulvibazar, and Habiganj face persistent problems from diseases such as canker, black spot, scab, and greening (Huanglongbing) [2, 3]. When infections are not caught early, fruit yield drops and cosmetic blemishes lower market prices [4, 5, 3].

Traditionally, disease checking has relied on manual scouting by agricultural officers or farmers, which is slow, labor-intensive, and prone to error over large acreages [6, 7]. Computer vision gives an automated alternative by processing RGB field images directly [8, 9]. CNNs have shown strong capability in learning discriminative visual features from plant images, with architectures like AlexNet, VGG16, VGG19, InceptionV3, ResNet, and DenseNet showing high accuracy on botanical datasets [2].

1.1 Context and Background

In Bangladesh, many rural households depend on seasonal crop and fruit harvests for income [10, 11], with over 39,678 acres

allocated for crop production nationwide [12, 10]. Because field environments are vulnerable to sudden microclimatic shifts and pest-borne infections, timely diagnosis is essential to guide targeted spraying and reduce pesticide waste [3, 9].

1.2 Research Motivation

Several studies have demonstrated high accuracy using deep CNNs for plant disease detection [5, 2, 11]. However, many rely on heavy networks (e.g., AlexNet or VGG19) that demand high computational throughput, large GPU memory, and long training times [2]. In practical agricultural settings, diagnostic software must often run on portable smartphones, microcontroller modules, or drones with constrained battery and memory. Few studies measure runtime latency, storage footprint, or memory consumption alongside accuracy. This paper addresses that gap by testing both diagnostic metrics and computational overhead across four pre-trained networks.

1.3 Objectives and Scope

The objectives of this work are:

- Fine-tuning four pre-trained CNN models (MobileNetV2, InceptionV3, ResNet50, and VGG19) using transfer learning on 1,463 citrus fruit images across four distinct conditions.
- Measuring classification metrics (accuracy, precision, recall, F1-score) and hardware-oriented parameters (training time, test inference latency, parameter count, RAM usage).
- Identifying the most viable model architecture for deployment on portable, battery-powered agricultural hardware.

1.4 Significance of Research

This research is done to provide concrete engineering benchmarks that link diagnostic accuracy directly to embedded hardware constraints in citrus disease detection. The data can serve as reference points for building field-ready mobile apps and drone-based scouting systems.
The structure of this paper is as follows. Section 2 presents the literature review. Section 3 discusses the methodology and model configurations. Section 4 covers the performance evaluation and complexity benchmarks. Section 5 presents the conclusions and future directions.

2. LITERATURE REVIEW

The people of Bangladesh heavily depend on agriculture. According to available statistics, about 87% of the rural population directly or indirectly depends on agricultural activities [13]. Because of this, early detection of plant diseases using automated methods has attracted a lot of research attention.
Ahmed and Muqit [5] designed a deep neural network pipeline for detecting citrus fruit and leaf infections. They used 2,293 images from PlantVillage and got an accuracy of 94.55%. Balafas et al. [8] did a comparative study between traditional machine learning models (SVM, Random Forest, SGD) and deeper networks (InceptionV3, VGG16, VGG19). Their results showed that deep features handle noisy backgrounds and different lighting conditions better than shallow hand-crafted descriptors.
For identifying diseases in citrus, Elaraby et al. [2] used a deep learning approach and achieved 94.0% accuracy with AlexNet and VGG19 backbones. Sujatha et al. [6] tested deep architectures for mobile plant pathology tools and pointed out the need for making diagnostics accessible at the field level. Hasan et al. [10, 22] applied transfer learning on regional crop datasets from Bangladesh, and they found that using pre-trained weights from ImageNet helped a lot with feature convergence on small target datasets. Table 1 gives a summary of recent related studies.

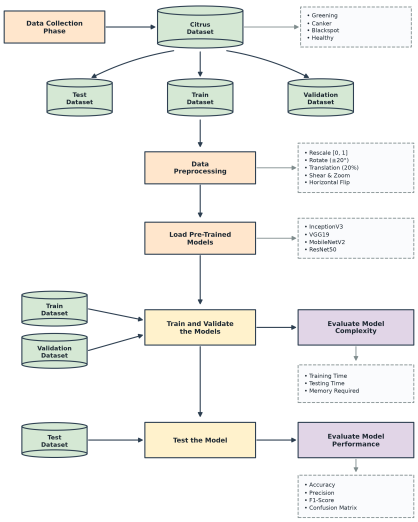

Fig. 1. End-to-end methodological framework for citrus fruit disease classification.

3. PROPOSED METHODOLOGY

The dataset was carefully collected and preprocessed to make it suitable for image processing. TensorFlow and Keras were used as the main framework for building and training the models. Four CNN architectures were selected for comparison: MobileNetV2, InceptionV3, ResNet50, and VGG19. The detailed workflow is shown in Figure 1.

3.1 Dataset Overview and Preparation

A dataset of 1,463 high-resolution RGB images has been collected from PlantVillage and Kaggle repositories. These images are sorted into four classes:

- **Canker**: Raised, rough lesions on the fruit surface caused by the bacterium *Xanthomonas citri*.
- **Black Spot**: Dark circular spots caused by the fungus *Phyllosticta citricarpa*.
- **Greening (HLB)**: Asymmetric yellowing of fruit and leaves, caused by the vector-borne bacterium *Candidatus* Liberibacter.
- **Healthy**: Normal fruit samples without any visible disease symptoms.

3.1.1 Class Distribution and Partitioning. The images were organized into folders by class and the counts are fairly balanced: 358 healthy, 387 greening, 362 black spot, and 356 canker. Figure 2 shows this distribution and Figure 3 displays some sample images from each class.
The dataset was split into 80% for training (1,170 images), 10% for validation (146 images), and 10% for testing (147 images). This split is shown in Figure 4. The test set was kept completely separate and only used after training was finished.

3.1.2 Data Pre-Processing and Augmentation. To prevent overfitting and make the models work better with different camera

Table 1. Summary of Related Literature on Citrus and Plant Disease Detection

No.	Author	Year	Class Types	Dataset Size	Classifier Architecture	Accuracy
1	R. Sujatha et al. [6]	2021	Black spot, Canker, Greening, Melanose, Healthy	609 images	LSVM, SGD, RF, InceptionV3, VGG16, VGG19	89.5%
2	Khattak et al. [12]	2021	Black spot, Canker, Scab, Greening, Melanose, Healthy	2293 (Fruits+Leaves)	Custom CNN + Softmax	94.55%
3	A. Elaraby et al. [2]	2022	Anthracnose, Black spot, Canker, Scab, Greening, Melanose	759 (Fruits+Leaves)	AlexNet, VGG19	94.0%
4	S. Parez et al. [14]	2023	Multiple plant pathological conditions	Plant leaves	Lightweight CNN + Softmax	99.0%
5	Dehuan Luo et al. [15]	2023	Anthracnose, Canker, Melanosis, Scab, Brown Spot, Leaf Miner	3202 images	Light-SA YOLOv8	92.6%
6	Madhusudan G. et al. [16]	2023	Black spot, Melanose, Canker, Greening, Healthy	609 RGB images	CNN, ResNet152V2, DenseNet121, InceptionResNetV2	98.0%
7	Poonam Dhiman et al. [17]	2023	HLB, Black spot, Melanose, Canker, Scab	2950 images	CNN + LSTM	98.25%
8	W. Gómez-Flores et al. [18]	2024	HLB, Greasy spot, Deficiencies, Citrus Mite, Red scale	953 images	DNN, Transfer Learning	Not reported
9	Nouman Butt et al. [19]	2024	Canker, Citrus scab, Black spot	2184 images	SVM, KNN	99.6%
10	J. Shermila et al. [20]	2024	Fruit blight, Greening, Scab, Melanoses	759 images	Custom CNN + LSTM	96.0%
11	A.K. Saini et al. [21]	2024	Anthracnose, Melanose, Brown Spot	759 images	SADCNN-ORBM	99.76%

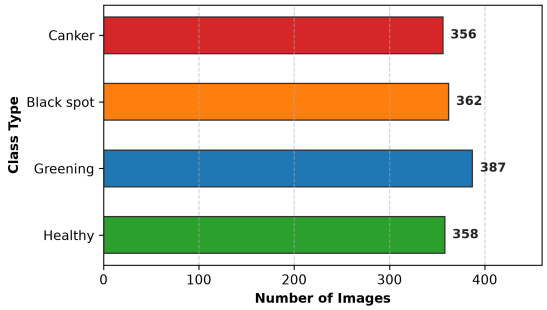

Fig. 2. Distribution of image samples across the four citrus fruit classes.

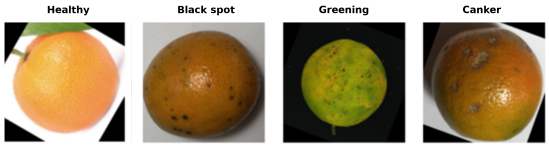

Fig. 3. Representative sample images from the experimental citrus dataset.

angles and lighting conditions, the following augmentations were applied during training:

- Rescaling pixel values into the range $[0, 1]$.
- Random rotation up to $\pm 20°$.
- Horizontal and vertical translation up to 20%.
- Random shearing of images.
- Random zoom up to 20%.
- Random horizontal flipping.

All images were resized to $224 \times 224 \times 3$ pixels and loaded in mini-batches of 32.

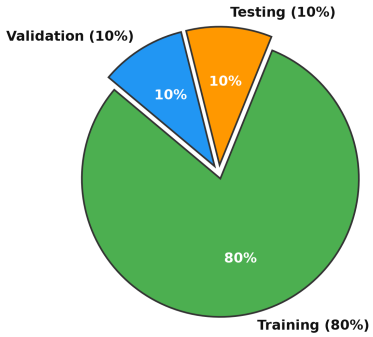

Fig. 4. Dataset partitioning into training (80%), validation (10%), and test (10%) subsets.

3.2 Transfer Learning Framework

In deep learning, transfer learning is a method where a model trained for one task is used as a starting point to solve another related problem [23, 22]. This approach speeds up learning and gives better results even on small datasets because the model already knows how to extract useful features from images. Transfer learning allows using pre-trained models which show good accuracy even on medium-sized datasets. Figure 5 shows how this process works.

Four pre-trained models were adapted for the four-class citrus disease task. The base convolutional feature extractors were frozen to retain ImageNet weights, and custom top classification heads were added in each case:

- **VGG19**: For the VGG19 backbone, the pre-trained feature extractor consists of 16 convolutional layers using uniform 3×3 filters across 5 pooling blocks [24, 25, 26]. The base weights were kept non-trainable during training. A 40% Dropout layer was added right after the base [27, 28, 29], followed by a Flatten layer [30], a Dense layer with 512 ReLU units, a second

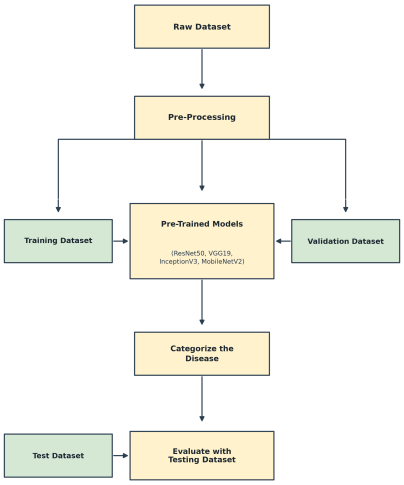

Fig. 5. Flowchart of the transfer learning fine-tuning pipeline.

Dropout layer (20%), and a final 4-unit Softmax classification layer [31]. Table 2 summarizes the full parameter breakdown.

Table 2. Layer Architecture and Parameter Details of Fine-Tuned VGG19

Layer (Type)	Output Shape	Parameters
VGG19 (Functional Base)	(None, 7, 7, 512)	20,024,384
dropout_1 (Dropout 40%)	(None, 7, 7, 512)	0
flatten (Flatten)	(None, 25088)	0
dense_1 (Dense, ReLU)	(None, 512)	12,845,056
dropout_2 (Dropout 20%)	(None, 512)	0
dense_2 (Dense, Softmax)	(None, 4)	2,052
Total Parameters:		32,871,492
Trainable Parameters:		12,847,108
Non-Trainable Parameters:		20,024,384

- **MobileNetV2**: As a lightweight model for edge deployment, MobileNetV2 relies on depthwise separable convolutions and inverted residual bottlenecks with linear activation units [32]. Freezing the base network keeps the low-level visual representations intact while significantly cutting down the number of trainable operations. The custom top head consists of 40% dropout, a flatten layer, 512 dense ReLU units, 20% dropout, and a 4-class softmax layer. Table 3 details the layer architecture.

Table 3. Layer Architecture and Parameter Details of Fine-Tuned MobileNetV2

Layer (Type)	Output Shape	Parameters
MobileNetV2 (Base)	(None, 7, 7, 1280)	2,257,984
dropout_1 (Dropout 40%)	(None, 7, 7, 1280)	0
flatten (Flatten)	(None, 62720)	0
dense_1 (Dense, ReLU)	(None, 512)	32,085,760
dropout_2 (Dropout 20%)	(None, 512)	0
dense_2 (Dense, Softmax)	(None, 4)	2,052
Total Parameters:		34,345,796
Trainable Parameters:		32,087,812
Non-Trainable Parameters:		2,257,984

- **InceptionV3**: Multi-scale spatial features are processed in InceptionV3 by factorized convolutions and asymmetric filtering inside parallel branches. The top classification layers were replaced with a flatten operation, 512 dense ReLU units, 20% dropout regularization, and a 4-unit softmax output. Table 4 presents the complete layer configuration.

Table 4. Layer Architecture and Parameter Details of Fine-Tuned InceptionV3

Layer (Type)	Output Shape	Parameters
InceptionV3 (Base)	(None, 5, 5, 2048)	21,802,784
flatten (Flatten)	(None, 51200)	0
dense_1 (Dense, ReLU)	(None, 512)	26,214,912
dropout_2 (Dropout 20%)	(None, 512)	0
dense_2 (Dense, Softmax)	(None, 4)	2,052
Total Parameters:		48,019,748
Trainable Parameters:		26,216,964
Non-Trainable Parameters:		21,802,784

- **ResNet50**: Residual shortcut connections are the core element of ResNet50, enabling direct gradient propagation across 50 layers to mitigate vanishing gradients. With the pre-trained convolutional base frozen, the classification head was built using a flatten layer, a 512-unit dense ReLU layer, 50% dropout, and a 4-class softmax output layer. Adam optimization was configured at a learning rate of 0.0001. Table 5 outlines the resulting parameter distribution.

Table 5. Layer Architecture and Parameter Details of Fine-Tuned ResNet50

Layer (Type)	Output Shape	Parameters
ResNet50 (Base)	(None, 5, 5, 2048)	23,587,712
flatten (Flatten)	(None, 100352)	0
dense_1 (Dense, ReLU)	(None, 512)	51,380,736
dropout_2 (Dropout 50%)	(None, 512)	0
dense_2 (Dense, Softmax)	(None, 4)	2,052
Total Parameters:		74,970,500
Trainable Parameters:		51,382,788
Non-Trainable Parameters:		23,587,712

3.2.1 Framework Implementation. All experiments were done in Python using TensorFlow and Keras. TensorFlow is a well-known open-source machine learning framework developed by the Google Brain Team. It provides a good environment for building and deploying machine learning models, and works well on CPUs, GPUs, and TPUs. The end-to-end framework, from data loading to GPU training to model export for mobile deployment (TensorFlow Lite, TensorFlow Serving), is shown in Figure 6.

3.3 Optimization and Training Configuration

All four models were trained using the Adam optimizer, which is an adaptive learning rate algorithm that combines methods from both AdaGrad and RMSProp. The learning rate was set to $\eta = 0.0001$. The loss function used was categorical cross-entropy, which is suitable for multi-class classification problems:

$$\mathcal{L}_{CCE} = -\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c}\log(\hat{y}_{i,c}) \quad (1)$$

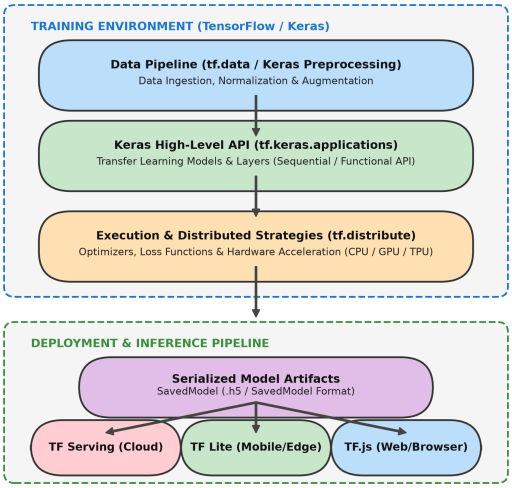

Fig. 6. Framework architecture covering training, serialization, and deployment pipelines.

where N is the mini-batch size, $C = 4$ is the number of classes, $y_{i,c} \in \{0, 1\}$ is the true label, and $\hat{y}_{i,c}$ is the predicted probability for class c. The models were trained for 50 epochs and the best weights were saved based on validation loss.

3.4 Evaluation Metrics

3.4.1 Performance Metrics. For evaluating how well the models performed, some standard parameters were used. The metrics are as follows:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{2}$$

$$\text{Precision} = \frac{TP}{TP + FP} \tag{3}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{4}$$

$$F1\text{-Score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{5}$$

where TP = True Positive, TN = True Negative, FP = False Positive, and FN = False Negative.

3.4.2 Computational Complexity Metrics. The complexity analysis of the models was done by measuring four things: total training time in seconds, inference time on the 147 test images in seconds, trainable and non-trainable parameter count, and how much memory each model uses in MB.

4. RESULT AND DISCUSSION

This section gives a detailed discussion of the results from the experiments. The models are compared using both performance analysis and complexity analysis.

4.1 Performance Evaluation

The performance analysis was done using precision, recall, and F1-score on the 147-image test set. In this case, MobileNetV2 showed the highest precision at 98.56%, recall at 98.54%, and F1-score at 98.54%. InceptionV3 was close behind with 97.21% precision, 97.30% recall, and 97.19% F1-score. VGG19 got 84.82% precision and 81.17% F1-score. On the other hand, ResNet50 showed the lowest performance in all metrics, with only 58.21% F1-score. Table 6 presents these results.

Table 6. Performance Comparison Across Fine-Tuned Models on the Unseen Test Set

Model Name	MobileNetV2	InceptionV3	ResNet50	VGG19
Precision (%)	98.56	97.21	66.66	84.82
Recall (%)	98.54	97.30	62.02	81.31
F1-Score (%)	98.54	97.19	58.21	81.17

4.1.1 Accuracy and Loss Analysis. Each model's accuracy and loss during training, validation, and testing have been compared. High accuracy means the model is learning the data well, while low loss means good performance. If the training loss is very low but the validation loss stays high, it can indicate overfitting where the model is learning noise instead of real patterns. The training accuracy of MobileNetV2 and InceptionV3 is better than the other models, and this remains the same for validation and test accuracy. MobileNetV2 produced the lowest test loss at 5.29%. ResNet50 had the highest loss value at 76.16%, which suggests that the residual blocks may need deeper layer unfreezing or a different learning rate to work well on this particular dataset. Table 7 presents these numbers.

Table 7. Accuracy and Loss Metrics Across Training, Validation, and Test Phases

Model	Training		Validation		Testing	
	Acc. (%)	Loss (%)	Acc. (%)	Loss (%)	Acc. (%)	Loss (%)
MobileNetV2	95.80	10.97	94.55	12.80	97.29	5.29
InceptionV3	95.89	10.60	97.95	6.72	96.59	7.44
ResNet50	57.14	96.91	61.90	83.92	67.34	76.16
VGG19	75.10	60.10	87.75	32.46	88.44	37.76

4.1.2 Confusion Matrix Analysis. The confusion matrix gives an overview of how each model classified the test samples, which helps in seeing where the models made mistakes. Figure 7 shows the confusion matrices for all four models on the 147 test images. MobileNetV2 and InceptionV3 classified every greening sample correctly and made very few errors on the other classes. ResNet50 had noticeable misclassifications between healthy and diseased fruit.

4.2 Computational Complexity Analysis

The complexity analysis is an important factor in evaluating these models, especially for deployment on devices with limited resources. Training time, inference time, and memory usage were measured for each model. Table 8 compares these values.

Table 8. Quantitative Computational Complexity Metrics Across Models

Metric	MobileNetV2	InceptionV3	ResNet50	VGG19
Training Time (s)	406.29	717.00	1040.00	3360.14
Inference Latency (s)	83.67	85.73	84.11	80.34
Memory Footprint (MB)	9.82	22.81	34.28	118.59

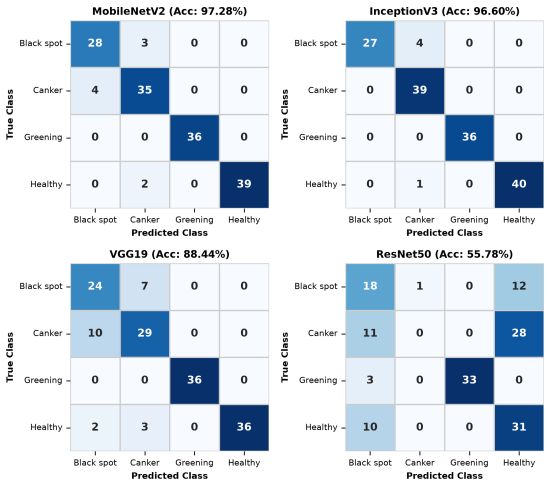

Fig. 7. Multi-class confusion matrices across all four fine-tuned CNN architectures on the test partition.

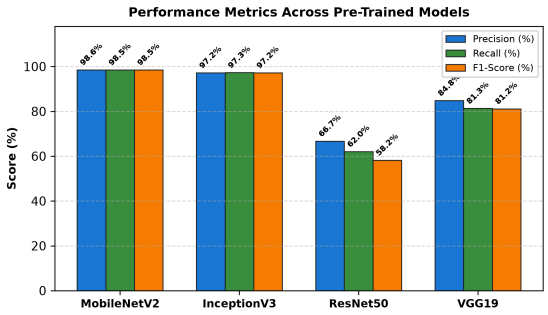

Fig. 8. Comparative diagnostic metrics across the four pre-trained architectures.

MobileNetV2 is the most efficient model with the shortest training time of 406.29 s and the lowest memory usage of 9.82 MB. Compared to VGG19, which took 3360.14 s to train and used 118.59 MB of memory, MobileNetV2 reduced training time by about 88% and memory usage by around 91.7%. The inference times were relatively close across all four models, ranging between 80.34 s and 85.73 s for the 147 test images, with VGG19 slightly faster at 80.34 s. Figure 8 shows a graphical comparison of all the metrics.

4.3 Comparative Discussion

This section gives a comparative discussion of the models used for citrus fruit disease diagnosis based on their performance and complexity metrics. In this work, the goal was to find the best model by looking at both accuracy and computational cost. The performance was evaluated using precision, recall, F1-score, and accuracy.
MobileNetV2 and InceptionV3 both had high training accuracy and low loss, showing good generalization. ResNet50 had much lower accuracy, which indicates that it did not learn the features well with frozen base layers. VGG19 showed decent results but was very heavy in terms of computation and memory.

After comparing all the results, MobileNetV2 turned out to be the best choice because it gives the highest accuracy, uses the least memory, trains the fastest, and is lightweight enough to be deployed on smartphones, portable scanners, and drone cameras for real-time disease monitoring in the field.

5. CONCLUSION

This study compared four pre-trained deep learning architectures (MobileNetV2, InceptionV3, ResNet50, and VGG19) on a four-class citrus fruit disease dataset of 1,463 images. Experimental results showed that MobileNetV2 achieved the strongest overall balance of classification and operational efficiency, recording 97.29% test accuracy, 98.54% F1-score, a rapid 406.29 s training duration, and a minimal 9.82 MB memory footprint. InceptionV3 offered competitive accuracy at 96.59% but incurred higher memory demands (22.81 MB). In contrast, ResNet50 showed lower classification efficacy (67.34% test accuracy, 58.21% F1-score) under frozen-base fine-tuning, while VGG19 required substantial training time (3360.14 s) and memory (118.59 MB). These results demonstrate that lightweight models using depthwise separable convolutions are well suited for direct deployment on mobile devices, portable field scanners, and drone platforms for automated citrus crop monitoring.

5.1 Limitations

This study has some limitations. The main one is that the dataset has only 1,463 images, which is relatively small for training deep learning models. Also, the images come from only two sources (PlantVillage and Kaggle), so the models were not tested under real field conditions with variable lighting, partial occlusions, or overlapping symptoms that occur in actual orchards.

5.2 Future Scope

Future work can explore several directions:

- Testing bio-inspired optimization algorithms like Grey Wolf Optimizer (GWO) or Particle Swarm Optimization (PSO) for automated hyperparameter tuning.
- Building hybrid models that combine the strengths of different architectures to get better accuracy and robustness.
- Adding explainable AI techniques such as Grad-CAM to highlight the disease regions in the image, which would help agricultural workers understand and trust the model predictions.

6. REFERENCES

[1] Research Square. Research square: Accelerating the dissemination of research, 2025. Accessed: 2 March 2025.

[2] Ahmed Elaraby, Walid Hamdy, and Saad Alanazi. Classification of citrus diseases using optimization deep learning approach. *Computational Intelligence and Neuroscience*, 2022:1–10, February 2022.

[3] ResearchGate. Researchgate: Professional network for scientists, 2025. Accessed: 2 March 2025.

[4] Ahmed Abdelmoamen Ahmed and Gopireddy Harshavardhan Reddy. A mobile-based system for detecting plant leaf diseases using deep learning. *AgriEngineering*, 3(3):478–493, July 2021.

[5] T. Ahmed, A. Muqit, J. Datta, M. Haque, and M. K. Haque. Prevalence and severity of different citrus diseases in sylhet

region. *Journal of Bioscience and Agriculture Research*, 23(2):1957–1968, 2020.

[6] R. Sujatha, Jyotir Moy Chatterjee, NZ Jhanjhi, and Sarfraz Nawaz Brohi. Performance of deep learning vs machine learning in plant leaf disease detection. *Microprocessors and Microsystems*, 80:103615, February 2021.

[7] Vinay Kukreja, Rishabh Sharma, and Rishika Yadav. Improving citrus farming through efficient and accurate diagnosis of lemon citrus canker disease with deep learning. In *2023 4th International Conference for Emerging Technology (INCET)*, pages 1–5. IEEE, May 2023.

[8] Vasileios Balafas, Emmanouil Karantoumanis, Malamati Louta, and Nikolaos Ploskas. Machine learning and deep learning for plant disease classification and detection. *IEEE Access*, 11:114352–114377, 2023.

[9] Hindawi. Hindawi: Open access publisher, 2025. Accessed: 2 March 2025.

[10] Tareq Hasan, Marjuk Ahmed Siddiki, and Md Naim Hossain. Detection of bangladeshi-produced plant disease using a transfer learning based on deep neural model. *Journal of Computer Science and Technology Studies*, 5(3):55–69, August 2023.

[11] Nafisha Binte Moin, Nabila Islam, Shamima Sultana, Lubaba Alam Chhoa, S. M. Ruhul Kabir Howlader, and Shamim H. Ripon. Disease detection of bangladeshi crops using image processing and deep learning - a comparative analysis. In *2022 2nd International Conference on Intelligent Technologies (CONIT)*, pages 1–8. IEEE, June 2022.

[12] Asad Khattak, Muhammad Usama Asghar, Ulfat Batool, Muhammad Zubair Asghar, Hayat Ullah, Mabrook Al-Rakhami, and Abdu Gumaei. Automatic detection of citrus fruit and leaves diseases using deep neural network model. *IEEE Access*, 9:112942–112954, 2021.

[13] Muhammad Sharif, Muhammad Attique Khan, Zahid Iqbal, Muhammad Faisal Azam, M. Ikram Ullah Lali, and Muhammad Younus Javed. Detection and classification of citrus diseases in agriculture based on optimized weighted segmentation and feature selection. *Computers and Electronics in Agriculture*, 150:220–234, July 2018.

[14] Sana Parez, Naqqash Dilshad, Turki M. Alanazi, and Jong Weon Lee. Towards sustainable agricultural systems: A lightweight deep learning model for plant disease detection. *Computer Systems Science and Engineering*, 47(1):515–536, 2023.

[15] Dehuan Luo, Yueju Xue, Xinru Deng, Bin Yang, Haifei Chen, and Zhujiang Mo. Citrus diseases and pests detection model based on self-attention yolov8. *IEEE Access*, 11:139872–139881, 2023.

[16] Madhusudan G. Lanjewar and Jivan S. Parab. Cnn and transfer learning methods with augmentation for citrus leaf diseases detection using paas cloud on mobile. *Multimedia Tools and Applications*, 83(11):31733–31758, September 2023.

[17] Poonam Dhiman, Amandeep Kaur, Yasir Hamid, Eatedal Alabdulkreem, Hela Elmannai, and Nedal Ababneh. Smart disease detection system for citrus fruits using deep learning with edge computing. *Sustainability*, 15(5):4576, March 2023.

[18] Wilfrido Gómez-Flores, Juan José Garza-Saldaña, and Sóstenes Edmundo Varela-Fuentes. Citrusuat: A dataset of orange citrus sinensis leaves for abnormality detection using image analysis techniques. *Data in Brief*, 52:109908, February 2024.

[19] Nouman Butt et al. Citrus disease detection and classification using machine learning techniques. *Journal of Computing & Biomedical Informatics*, 6(02), March 2024.

[20] Josephin Shermila P, Akila Victor, S. Oswalt Manoj, and E. Anna Devi. Automatic detection and classification of disease in citrus fruit and leaves using a customized cnn based model. *Boletin Latinoamericano y del Caribe de Plantas Medicinales y Aromaticas*, 23(2):180–198, March 2024.

[21] Ashok Kumar Saini, Roheet Bhatnagar, and Devesh Kumar Srivastava. Sadcnn-orbm: a hybrid deep learning model based citrus disease detection and classification. *International Journal of Electrical and Computer Engineering (IJECE)*, 14(2):2191, April 2024.

[22] Al-Kindi Publisher. Al-kindi publisher: Open access journals and books, 2025. Accessed: 2 March 2025.

[23] Ebin.pub. Ebin.pub: Free ebooks and digital resources, 2025. Accessed: 2 March 2025.

[24] Ambreen Shah, Muhammad Attique Khan, Ahmed Ibrahim Alzahrani, Nasser Alalwan, Ameer Hamza, Suresh Manic, Yudong Zhang, and Robertas Damaševičius. Fuzzyshallow: A framework of deep shallow neural networks and modified tree growth optimization for agriculture land cover and fruit disease recognition from remote sensing and digital imaging. *Measurement*, 237:115224, September 2024.

[25] Journal of Theoretical and Applied Information Technology. Official website of jatit. `http://www.jatit.org`, 2025. Accessed: 3 March 2025.

[26] Staffordshire University ePrints Repository. Staffordshire university eprints repository. `https://eprints.staffs.ac.uk`, 2025. Accessed: 3 March 2025.

[27] Frontiers. Frontiers: Peer-reviewed research articles. `https://www.frontiersin.org`, 2025. Accessed: 3 March 2025.

[28] Patrick Rexrode Murphy. *Self-Care and Quality of Life of Remotely Monitored Appalachians with Heart Failure*. PhD thesis, West Virginia University Libraries, 2023.

[29] Tampere University Repository (Trepo). Trepo: Open institutional repository of tampere university. `https://trepo.tuni.fi`, 2025. Accessed: 3 March 2025.

[30] Uppsala University DiVA Portal. Diva portal: Academic archive of uppsala university. `https://uu.diva-portal.org`, 2025. Accessed: 3 March 2025.

[31] PeerJ. Peerj: Open access research journal. `https://peerj.com`, 2025. Accessed: 3 March 2025.

[32] Ejaz Khan, Muhammad Zia Ur Rehman, Fawad Ahmed, and Muhammad Attique Khan. Classification of diseases in citrus fruits using squeezenet. In *2021 International Conference on Applied and Engineering Mathematics (ICAEM)*, pages 67–72. IEEE, August 2021.